Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Spacing Matters: Back-to-Back Replay Understates Tick-to-Trade Latency in a Reproducible C++ Market-Data Pipeline

RAJVARDHAN JADHAV[1]
[1]Fox School of Business, Temple University, Philadelphia, PA, USA (e-mail: rajvardhanjadhav124@gmail.com)
Corresponding author: Rajvardhan Jadhav (e-mail: rajvardhanjadhav124@gmail.com).

Funding information to be confirmed by the author.

ABSTRACT Latency of software trading systems is usually measured by replaying recorded market data packets back to back, a practice that keeps processor caches and branch predictors warm. We study how much this understates the cost of a tick when packets are spaced as on a real feed. Using an open, reproducible C++20 pipeline that covers feed arbitration, an order book, a market-making decision, risk checks, order management and order staging, we insert an idle gap before each tick, outside the timed interval and with no memory traffic, and compare against a matched back-to-back control. On a shared two-core cloud virtual machine, the median latency of ticks that send no order rises by a factor of 2.9 at a gap of 100 microseconds, 5 at one millisecond and 14 at ten milliseconds. Running the pipeline's code on a shadow instance during the gap recovers only part of the penalty, so the cause is not isolated. We replicate the back-to-back timings on a second host, compare the order book with a standard-library reference, and report paired synthetic strategy ablations with multiplicity-corrected inference. The full-session replay is verified byte for byte against the backtest before any latency is reported. We make no claim about exchange latency, real fills or profitability. All code, raw outputs and scripts regenerate the reported numbers.

INDEX TERMS Benchmarking methodology, cache warming, latency measurement, limit order book, reproducibility, tick-to-trade

I. INTRODUCTION

TICK-TO-TRADE latency, the time from the arrival of a market data packet to the staging of an order in response, is a central figure of merit for electronic trading software. Published and informal figures are frequently obtained by replaying a captured session through the system as fast as it will accept packets. This keeps instruction and data caches and branch predictors warm and so yields a best case. A live system sees a packet every few hundred microseconds to tens of milliseconds, and between packets the same core may sit idle or run other code. How much the best-case figure understates the cost of a tick at realistic spacing is rarely reported together with an artifact that lets a reader check it.

This article makes three contributions, each tied to preserved raw outputs.

1) **A controlled spacing measurement.** In a pinned, single-thread replay of a byte-verified session, an idle gap before each tick increases measured latency by large factors without the benchmark evicting its own working set, relative to a matched back-to-back control (Section V).
2) **A test of one mitigation.** Running the pipeline's code path on a shadow instance during the gap recovers only part of the penalty, which narrows but does not settle the cause (Section VI).
3) **A reproducible artifact and honest scope.** The pipeline, tests, experiment drivers and manuscript generator rebuild from one command. Hot-cache timings were replicated on a second host, and a book-update baseline and paired synthetic strategy ablations are reported with their limits (Sections VII and VIII). No profitability, formal-verification or exchange-latency claim is made.

The pipeline design combines familiar techniques. The novelty is the measurement and the evidence chain, not a new algorithm.

II. RELATED WORK

Order books and markets. Limit order book structure and dynamics are surveyed in [1]. Stochastic and queue-reactive

models of book dynamics appear in [2], [3], and the value of queue position is analyzed in [4]. Market making under inventory risk is treated classically in [5], and latency-driven market design in [6]. These works concern market models and incentives, not software latency measurement. We use them only to place the synthetic strategy experiments of Section VIII.

Benchmarking methodology. Measurement bias from seemingly irrelevant experimental setup was demonstrated in [7]. Statistically rigorous benchmarking with a bounded number of repetitions is treated in [8], and reporting practice for scientific benchmarking of parallel systems in [9]. Our spacing experiment is a concrete instance of this concern in a trading-system setting.

Cache warming in low-latency software. The phenomenon itself is known to practitioners. Because the order-sending path of a trading system executes rarely, its code and data are often cold when needed. The C++ low-latency design literature describes cache warming, for example running the engine on every event but releasing orders only when needed, and reports a large benefit in a cold-versus-warm microbenchmark [10]. That benchmark compares access patterns over a data set rather than sweeping idle time between invocations. Open-source matching engines such as [11] report latency and throughput tests. In the sources we could access, we did not find a controlled sweep of idle time between ticks, with a matched back-to-back control, for an inspectable end-to-end pipeline. This is a statement about our search and not a claim of priority (Section X).

III. SYSTEM UNDER STUDY

A. PIPELINE

An encoded feed packet passes through arbitration between two redundant lines, gap detection and snapshot recovery, an order-book update, a market-making decision, pre-trade risk checks, order management and order-entry staging, in one thread run to completion. The book uses flat price levels, pooled intrusive order nodes, a bitmap of occupied levels and an identifier lookup table. The node size of 32 bytes is a compile-time check and not a proof of cache behavior. A separate compile target checks selected hot-path headers with exceptions and run-time type information disabled. The measured path excludes the network interface, the kernel network stack, transport security, message parsing and the exchange. Correctness checks are differential, using an independent reference book and a replay hash, and are not formal verification.

B. STRATEGY AND SYNTHETIC VENUE

The strategy combines a quoted spread, an order-book imbalance lean and an inventory skew with risk limits. Its coefficients are engineering choices and no optimal-control claim is made. The backtester stages orders through the production gateway and order manager and executes them in a queue-aware synthetic venue with separate market-data, ingress and egress delays. The generator includes informed flow that the imbalance reveals, so removing the imbalance lean tests whether the engineered signal is used and not whether such a signal exists in historical data. Marked profit and loss is decomposed into spread edge, inventory mark-to-market and explicit fees. There is no modeled endogenous market impact and no calibrated real-venue queue model.

IV. MEASUREMENT METHOD

Timed interval. Each tick is timed with ordered reads of the processor time-stamp counter around ingest and decision. The median timer overhead, about 14 to 15 ns, is subtracted. A tick is classified as an *order* tick if at least one order frame is staged and as a *no-order* tick otherwise (tick-to-decision). Latencies are stored in a fixed-bin histogram that preserves bin counts and tick calibration, and quantiles use the nearest-rank definition $\lceil qn \rceil$. A defect in the predecessor used $\lfloor qn \rfloor$, which can fall below the stated quantile for odd sample counts, and was corrected with regression tests.

Verification gate. Before any latency is printed, the production pipeline must emit the exact wire bytes of the backtest session, checked by hash. A failed match aborts the run. The gapped passes replay a prefix of the same session through the same code path and are not re-hashed themselves.

Hosts. Host A is a shared AMD EPYC 9V74 virtual machine with a four-core quota and GCC 14. Host B is a shared virtual machine with an Intel Xeon @ 2.80GHz processor, two virtual cores and GCC 13.3. Neither has hardware performance counters, processor isolation or a fixed frequency. Both are cloud virtual machines of the same provider class, so the replication tests software and compiler variation and not hardware diversity. The benchmark thread is pinned to one core.

Spacing control. For each gap g, a spin loop of duration g runs before every tick, outside the timed interval. Its latency is compared with a gap-0 pass over the same first N inputs, so the only difference between the two is idle time between ticks. For large g the number of inputs N is reduced so that a pass takes about two seconds ($N = 40{,}000$ for $g \leq 20$ µs, 20,000 at 100 µs, 2,000 at 1 ms and 300 at 10 ms). The spin executes no memory traffic and so does not evict the benchmark's own working set. Ratios are computed per run, and the median over five sequential runs is reported.

Shadow warming. In the warmed variant, a second, independent pipeline instance fed its own copy of the session from the first input processes five inputs in the last 3 µs of each gap. The same code paths therefore run without touching the live book's data and without sending anything.

V. SPACING RESULTS

Table 1 and Fig. 1 give the median over five runs on Host B. The penalty is negligible at a gap of 1 µs, about 8% at 5 µs, and grows steadily thereafter. By 1 ms the no-order p50 is five times its back-to-back value and the order p50 has grown more than tenfold. The no-order p99 also rises, from 0.24 µs back to back to about 1.1 µs at 100 µs and 2.0 µs at 1 ms. At the 10 ms gap the five-run ratio ranged from 10 to 18 because only

TABLE 1. Median over five runs on Host B. No-order p50 (the median latency of ticks that send no order), its ratio to the matched gap-0 control, the same median with shadow warming, and the order-tick p50. Order-tick values at gaps of 1 ms and above rest on few samples (37 and 12 per run) and are descriptive. The 10 ms row uses only 300 inputs. Source: `results/spacing/*/benchmark_*.json`.

Gap (µs)	N	No-order p50 (ns)	vs. control	Warmed (ns)	Order p50 (ns)
1	40000	98	1.0×	–	177
5	40000	105	1.1×	107	234
20	40000	164	1.7×	188	328
100	20000	291	2.9×	254	634
1000	2000	531	5.0×	394	1714
10000	300	2742	14.1×	1188	3291

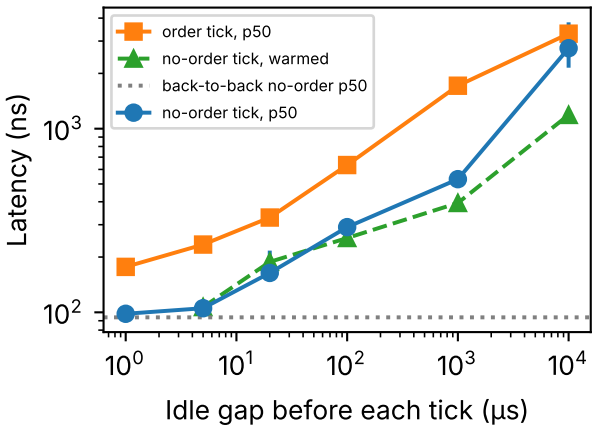


FIGURE 1. Latency versus idle gap before each tick on logarithmic axes. Bars show the minimum to maximum of the five-run p50. The dotted line is the back-to-back no-order p50.

274 no-order samples per run remain. For scale, a deliberately harsh cold-cache case, a 4 MiB walk before every tick over the first 40,000 inputs, gave a no-order p50 of 708 to 720 ns and an order p50 of 1.42 to 1.55 µs over five runs. The idle-gap result cannot be attributed to the benchmark's own eviction because the spin touches no memory.

VI. WARMING TEST AND WHAT IS NOT SHOWN

The *Warmed* column of Table 1 gives the no-order p50 with shadow warming. Warming recovers part of the penalty only at long gaps (1 ms: from 531 to 394 ns; 10 ms: from 2.74 to 1.19 µs) and none at 20 and 100 µs. Code-path warming alone therefore does not remove the effect. The remaining penalty is consistent with decay of the live book's data in cache, or with predictor state specific to that data, but this was not isolated.

State decay in caches, branch predictors or translation buffers, descheduling or noisy neighbors on the shared virtual machine, and frequency or idle-state effects are all consistent with the data. The p50 is robust to rare descheduling, which argues against pure scheduler noise, but a bare-metal replication with performance counters is required to attribute the effect. The spin loop is itself code and could perturb predictor or frequency state.

TABLE 2. Paired synthetic profit-and-loss differences versus the base strategy, in generator currency units over the simulated episode. Wins and losses count markets. The Holm p is the adjusted exact sign-test probability. Source: `paired_statistics.csv`.

Study	Variant	Paired difference [95% CI]	Wins/losses	Holm p	Max pos.
primary	noalpha	-16.78 [-17.53, -16.03]	0/16	0.000122	12
primary	wide	-17.27 [-18.18, -16.36]	0/16	0.000122	7
primary	noskew	7.45 [1.80, 13.09]	11/5	0.21	40
primary	strongskew	-4.82 [-5.27, -4.38]	0/16	0.000122	11
confirmation	noalpha	-16.63 [-17.19, -16.07]	0/16	0.000122	10
confirmation	wide	-17.01 [-17.78, -16.23]	0/16	0.000122	10
confirmation	noskew	8.71 [3.72, 13.70]	14/2	0.00418	40
confirmation	strongskew	-4.81 [-5.18, -4.43]	0/16	0.000122	9

VII. CROSS-HOST REPLICATION AND BOOK BASELINE

Hot-cache order-tick p50 ranged 96.1–103.9 ns on Host A and 158.2–161.1 ns on Host B. The p99 ranged 140.4–164.8 ns versus 333.9–356.8 ns, and sustained throughput 16.08–20.37 versus 8.47–9.90 million inputs per second. Absolute figures therefore differ across hosts by roughly 1.6 times at the median and 2 times in throughput even though both are shared cloud virtual machines, so they are host-specific. They are also best-case, back-to-back numbers and should be read together with Section V.

An offline operation benchmark compares the flat book with an independent `std::map`/`std::list` reference over five seeded tapes of 200,000 timed mixed operations after 20,000 warm-up operations, with a target of 5,000 resting orders. Return values and periodic state digests must match before timing, and final digests must match. Execution order alternates across seeds. The median speed ratio was 4.87 on Host A and 4.58 on Host B, so this relative result was stable across hosts. This is a book-update baseline, not a comparison against a tuned third-party engine.

VIII. SYNTHETIC STRATEGY ABLATIONS

A. DESIGN AND STATISTICS

The primary run uses 16 synthetic markets (seeds 1000 to 1015) of 200,000 requests each, and a fixed confirmation run uses seeds 5000 to 5015. Variants remove the imbalance lean, widen the quotes, remove the inventory skew or strengthen it. All variants see the same markets. For paired differences d_i we compute the mean $\bar{d}$, the standard error $s_d/\sqrt{n}$ and a Student-t mean interval, together with exact two-sided binomial sign tests that exclude ties and market-level bootstrap intervals. The Holm procedure [12] controls the four-comparison family separately for the primary and confirmation experiments. Mean intervals and sign tests answer different questions and need not agree. Fills are not independent replicates, so paired markets are the sampling units.

B. PRIMARY AND CONFIRMATION RESULTS

Removing the imbalance lean or widening the quotes reduces simulated profit in all 16 markets in both runs, consistent with the informed flow built into the generator. Strengthening the inventory skew also reduces profit. Removing the inventory skew raises mean profit in both runs (differences of 7.45 and 8.71) while holding positions up to the cap of 40, against 7 to 12 for the other variants, so much of the gain is inventory

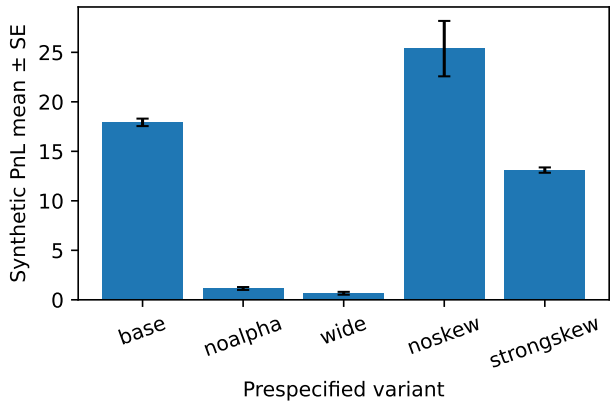


FIGURE 2. **Primary synthetic ablation means and standard errors across 16 markets. Source:** `primary.json`.

TABLE 3. **Mean paired difference versus the base in seven settings of the same generator, with wins and losses over eight markets in parentheses. These are not seven empirical datasets. With eight markets the smallest attainable Holm-adjusted sign-test probability in the 28-comparison family is 0.22, so no regime-level comparison is significant after correction. Source:** `regime_statistics.csv`.

Regime	No imbalance	Wide quotes	No skew	Strong skew
default	-3.39 (0/8)	-3.35 (0/8)	1.86 (3/5)	-0.97 (0/8)
volatile	-8.09 (0/8)	-7.52 (0/8)	4.41 (6/2)	-2.49 (0/8)
quiet	-1.52 (0/8)	-1.60 (0/8)	1.32 (5/3)	-0.40 (0/8)
informed	1.02 (5/3)	-1.16 (2/6)	-3.18 (3/5)	1.71 (7/1)
uninformed	-0.53 (0/8)	-0.55 (0/8)	2.03 (6/2)	-0.18 (0/8)
thin	-2.64 (0/8)	-3.12 (0/8)	-0.52 (2/6)	-0.25 (3/5)
slow	-3.37 (0/8)	-3.31 (0/8)	2.44 (4/4)	-1.01 (0/8)

exposure. Its mean interval excludes zero in both runs, but the sign test is not significant in the primary run after correction ($p = 0.21$) and is in the confirmation run ($p = 0.004$). This is an example of the disagreement between interval and sign-test questions noted above, and it shows why mean profit alone is not evidence of a better strategy.

C. REGIMES, DELAY, QUEUE AND FEES

Table 3 shows that widening the quotes is negative in all seven settings and that removing the imbalance lean is negative in six of seven and positive in the setting named informed. Removing the inventory skew changes sign across settings. Because eight markets cannot reach significance in a 28-comparison family, we report these as consistent in direction, not as established.

Table 4 gives the sensitivity results. The base strategy retains 61% of its profit at 100 µs per hop and 71% at a queue penalty of 100%. At a fee of 5 basis points per fill every strategy's profit falls (the base strategy's turns negative), so the synthetic profits are small relative to plausible fees. These are sensitivities of the simulator, not measurements of a real venue.

D. HELD-OUT SELECTION

A small grid over imbalance lean and inventory skew was ranked on training seeds 55000 to 55007 with an inventory penalty of one fixed before selection. The two retained candidates were run on held-out seeds 155000 to 155007. The best candidate on training data was the base configuration itself, so the held-out difference is zero in all eight markets. The other retained candidate lost 2.41 on average in all eight markets. The grid was not adjusted after inspecting held-out outputs. The holdout assesses seed generalization within one generator and not generalization to market data.

TABLE 4. **Mean synthetic profit and loss by variant under delay, queue-penalty and fee scenarios (eight markets of 40,000 requests). Delay applies to each of three hops, so 100 µs adds 300 µs across them before jitter. Source:** `sensitivity_statistics.csv`.

Setting	Base	No imbalance	Wide	No skew	Strong skew
Delay 0 µs per hop	3.65	0.23	0.08	5.46	2.60
Delay 5 µs per hop	3.55	0.23	0.14	5.76	2.61
Delay 20 µs per hop	3.32	0.18	0.15	5.42	2.48
Delay 50 µs per hop	2.88	0.08	0.12	3.60	2.30
Delay 100 µs per hop	2.24	0.00	0.24	2.95	2.05
Queue penalty 0%	3.80	0.25	0.08	6.21	2.62
Queue penalty 50%	3.17	0.19	0.08	4.84	2.33
Queue penalty 100%	2.71	0.31	0.09	4.19	2.16
Fee 1 bp	1.59	-0.65	0.05	3.37	0.70
Fee 5 bp	-7.17	-4.11	-0.52	-7.57	-7.63

IX. LIVE-FEED OBSERVATIONS

The repository documents a live paper-mode session on a public BTC-USD level-2 feed, with captures of 60 to 90 seconds. Replaying the same 60-second capture with different gaps between messages gave a per-message p50 of 18.4 µs back to back and 63.5 µs at the recorded spacing of about 57 ms, roughly 3.5 times. Spinning instead of sleeping between messages did not change this. That workload includes message parsing and conversion, which dominate (the pipeline itself costs about 1.3 µs per packet), so absolute values are not comparable to Table 1. We report it because the direction agrees. **These observations were taken by the author on a different shared two-core virtual machine (the repository's results page describes a 2.1 GHz Xeon for its runs of that period), the capture is not in the repository, and they were not reproduced by the pipeline of this article.** They are corroboration and not evidence.

X. DISCUSSION

Practical implication. A reader of a back-to-back tick-to-trade figure should ask what the latency is when ticks are spaced. We suggest reporting, alongside the back-to-back figure, the latency at one or more spaced gaps with a matched control, and stating the host, the sample counts and how the idle time is produced. The gap-sweep option added to the benchmark makes this a single command.

Threats to validity. The mechanism is measured and not explained, and the warming test narrows but does not settle the cause. Two shared cloud virtual machines with five sequential runs each, no isolation and no performance counters do not establish portability, and there is no bare-metal result. A spin loop is one model of idle time, and real feeds interleave other code and interrupts. Order-tick statistics at large gaps rest on few samples. The recorded session and strategy results

are synthetic. The map/list reference is deliberately simple. Several references were verified against bibliographic records and not full texts, and a systematic search of the low-latency systems literature for prior controlled spacing measurements was limited to the sources we could access. Nothing here measures network, interface, kernel or exchange latency, real fills, market impact or profitability.

XI. CONCLUSION

Back-to-back replay is a best case. In a verified, pinned pipeline on a shared virtual machine, adding idle time between ticks raised the median latency of ticks that send no order by 2.9 times at 100 µs, 5 times at 1 ms and 14 times at 10 ms, and running the same code path on a shadow instance recovered only part of that. The finding is a caution for how tick-to-trade latency is reported, not a claim about exchange latency. Bare-metal replication with performance counters and independent reproduction of the artifact are the most valuable next steps.

DATA AND CODE AVAILABILITY

The pipeline, tests, experiment drivers, raw outputs, hashes and manuscript generator are in the repository branch accompanying this article. The reproduction command is `bash research/reproduce.sh`, which builds, tests, runs the experiments, regenerates figures and tables and compiles the manuscript. The spacing study is `research/run_spacing_study.py` followed by `research/analyze_spacing.py`. No exchange credentials or orders are used.

ACKNOWLEDGMENT

Disclosure of AI use. The C++ source, tests, experiment scripts and portions of this article's text, tables and figures were produced with the assistance of Anthropic's Claude, including Claude Code, under the author's direction. [Author to confirm which sections used AI-generated content and at what level, as IEEE policy requires.] The author reviewed the results and is responsible for the content.

REFERENCES

[1] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," arXiv:1012.0349, 2010.
[2] R. Cont, S. Stoikov, and R. Talreja, "A stochastic model for order book dynamics," *Operations Research*, vol. 58, no. 3, pp. 549–563, 2010.
[3] W. Huang, C.-A. Lehalle, and M. Rosenbaum, "Simulating and analyzing order book data: The queue-reactive model," *Journal of the American Statistical Association*, 2015.
[4] C. C. Moallemi and K. Yuan, "A model for queue position valuation in a limit order book," 2017, working paper, Columbia Business School, revised June 2017.
[5] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, 2008.
[6] E. Budish, P. Cramton, and J. Shim, "The high-frequency trading arms race: Frequent batch auctions as a market design response," *The Quarterly Journal of Economics*, vol. 130, no. 4, pp. 1547–1621, 2015.
[7] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, pp. 265–276.
[8] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. 2013 Int. Symp. Memory Management (ISMM)*, 2013.
[9] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
[10] P. Bilokon and B. Gunduz, "C++ design patterns for low-latency applications including high-frequency trading," arXiv:2309.04259, 2023.
[11] I. Jericevich, D. Sing, and T. Gebbie, "CoinTossX: An open-source low-latency high-throughput matching engine," arXiv:2102.10925, 2021.
[12] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

RAJVARDHAN JADHAV holds a B.Tech. degree in computer science from the National Institute of Technology Warangal, India, and is pursuing the M.S. degree in quantitative finance and risk management at the Fox School of Business, Temple University, Philadelphia, PA, USA, with expected graduation in May 2027. His interests include quantitative research and risk modeling.